

RENCANA TUGAS MAHASISWA (RTM) - PROYEK AKHIR (TUGAS BESAR)

Mata Kuliah: Analisis Data untuk IoT (Semester 5) **Program Studi:** Teknologi Rekayasa Komputer (TRK)

1. Tujuan Tugas

Mahasiswa mampu merancang, mengonfigurasi (*deploy*), dan mempresentasikan arsitektur *end-to-end IoT Data Analytics* yang fungsional secara utuh untuk memecahkan masalah pemantauan di bidang pertanian pintar (*Smart Agriculture*) atau lingkungan.

2. Spesifikasi Arsitektur Wajib

Setiap kelompok diwajibkan untuk membangun sistem yang mengintegrasikan 5 (lima) lapis komponen utama berikut ini:

- Data Collector (Sensor Fisik / Simulator):** Sebuah program (*script* Python/C++) atau mikrokontroler fisik (misal: ESP32) yang bertugas membaca atau merakit angka (*generate*) metrik lingkungan (seperti suhu, kelembaban, kecepatan angin, dll.) lalu mengirimkannya ke jaringan.
- Message Broker (Jalur Komunikasi MQTT, AMQP, atau Valkey):** Terminal lalu lintas data *real-time*. Wajib mengimplementasikan protokol sesuai peruntukannya:
 - **MQTT:** Sangat ringan (*lightweight*), **wajib** digunakan untuk rute komunikasi dari **IoT Node (Data Collector)** ke **Message Broker**.
 - **AMQP atau Valkey Pub/Sub:** Digunakan untuk rute komunikasi internal jaringan (B2B) dari **Message Broker** ke **Aplikasi (Data Pipeline Worker)**. (Catatan: Valkey memiliki keunggulan karena dapat berperan ganda sebagai *message broker* sekaligus *database/cache in-memory*). Mahasiswa sangat disarankan menggunakan *broker* RabbitMQ (yang mendukung MQTT & AMQP sekaligus) atau kombinasi Mosquitto (MQTT) dengan Valkey.
- Data Pipeline Worker (Pekerja Pemroses Data):** Sebuah *script* Python (*subscriber/consumer*) yang berjalan terus-menerus di latar belakang (*daemon/worker*). Tugas utamanya meliputi:
 - Menangkap (*subscribe/consume*) aliran data mentah dari Message Broker (via MQTT, AMQP, atau Valkey).
 - Melakukan pra-pemrosesan (*cleaning*) otomatis secara kilat (misal: menambal data kosong NaN atau membuang batas nilai *outlier* yang mustahil).
 - Menjalankan model pendeteksi anomali dasar (*rule-based*).
 - Menyuntikkan (*insert*) data bersih tersebut ke dalam *Database*.
- Time-Series Database (Basis Data Runtun Waktu):** Penyimpanan data yang teroptimasi khusus untuk melahap ribuan data waktu secara efisien. Mahasiswa **wajib** memilih salah satu dari teknologi berikut:
 - **InfluxDB** (Sangat disarankan karena dirancang murni untuk IoT).
 - **MongoDB** (Dengan mengaktifkan fitur khusus *Time Series Collections*).
 - **PostgreSQL** (Dengan memasang ekstensi *TimescaleDB*).
- Analytic Dashboard (Panel Analisis Visual):** Antarmuka interaktif (*front-end*) bagi *user* untuk memantau data *real-time* maupun menganalisis tren historis mingguan/bulanan. Direkomendasikan menggunakan **Grafana** (karena kemudahan integrasi dengan *Time-series Database*), namun kelompok dibebaskan jika ingin menggunakan *dashboard* lain seperti **Metabase** atau merakit sendiri menggunakan **Streamlit**.

3. Uraian Pelaksanaan

- Metode Pengerjaan:**
 - Dikerjakan secara berkelompok (maksimal 3 orang per kelompok).
 - Pemilihan tema/studi kasus (misal: Monitoring Hidroponik, Monitoring Polusi PM2.5, Cuaca Ekstrem) diserahkan ke kreativitas mahasiswa, namun wajib mendapat persetujuan dosen pada **Pekan 9** (Segera setelah UTS).
- Tahapan Pengerjaan:**
 - **Pekan 10-11 (Infrastruktur):** Desain topologi jaringan, instalasi Message Broker, Database, dan *Dashboard* (sangat disarankan menggunakan *Docker Compose* agar seluruh sistem terangkai instan).
 - **Pekan 12-13 (Coding):** Pembuatan kode untuk *Data Collector* dan *Data Pipeline Worker*. Pengetesan aliran dari ujung-ke-ujung (*End-to-End Test*).
 - **Pekan 14-15 (Analitik & Alert):** Desain visual panel analitik (merakit metrik korelasi, histogram, *line chart*) dan pemasangan fitur *Alerting* (sistem akan mengirim pesan Telegram/Email jika *Worker* mendeteksi parameter bahaya). Finalisasi proyek.
 - **Pekan 16 (UAS):** Pengumpulan luaran dan Presentasi Akhir/Demonstrasi.

4. Luaran yang Dihasilkan

- **Kode Sumber (Source Code):** Tersimpan dengan struktur rapi di repositori GitHub, memuat file *script* Python, file *docker-compose.yml* (jika ada), serta panduan cara menjalankannya di *README.md*.
- **Demonstrasi Langsung (Live Demo):** Kelompok wajib mampu melakukan *live demo* saat ujian akhir, membuktikan bahwa sesaat setelah *Data Collector* dihidupkan, *Dashboard* di layar langsung bergerak secara waktu-nyata (*real-time*).

- **Laporan Proyek Akhir:** Berformat dokumen cetak/PDF, minimal 7 halaman (Format terlampir di berkas Format_Laporan_Tugas_Besar.md).